

📁 Projeto de Análise de Sentimentos

Projeto completo de Análise de Sentimentos utilizando o dataset **Sentiment140** com modelos clássicos (baseline) e modelos Transformer SOTA.

📁 Estrutura do Projeto

```
sentiment_analysis/
├── data/                                # Dados brutos e processados
│   ├── .gitkeep
│   └── sentiment140_processed.csv      # Dataset processado (gerado)
├── notebooks/                          # Jupyter notebooks
│   └── 1.0-EDA.ipynb                 # Análise Exploratória de Dados completa
├── src/                                # Código fonte modular
│   ├── __init__.py
│   ├── 1_data_loader.py              # Carregamento do dataset Sentiment140
│   ├── 2_preprocessing.py            # Limpeza e pré-processamento de texto
│   ├── 3_train_baseline.py           # Modelo baseline (TF-IDF + Logistic Regression)
│   ├── 4_train_transformer.py        # Fine-tuning de modelos Transformer (DistilBERT)
│   ├── 5_predict.py                  # Script de predição interativo
│   └── utils.py                       # Funções utilitárias
├── models/                            # Modelos treinados (gerado)
│   ├── baseline_model.pkl             # Modelo baseline salvo
│   └── sentiment_transformer/         # Modelo transformer salvo
├── results/                           # Resultados do treinamento (gerado)
├── logs/                              # Logs do treinamento (gerado)
├── .gitignore                         # Arquivos a ignorar no git
├── requirements.txt                   # Dependências do projeto
└── README.md                         # Este arquivo
```

📁 Início Rápido

1. Instalação das Dependências

```
pip install -r requirements.txt
```

Dependências principais:

- `pandas`, `numpy` - Manipulação de dados
- `scikit-learn` - Modelos de ML clássicos
- `nltk` - Processamento de linguagem natural
- `matplotlib`, `seaborn`, `wordcloud` - Visualizações
- `transformers`, `datasets` - Modelos Transformer e datasets Hugging Face
- `torch` - Backend para deep learning
- `jupyter` - Notebooks interativos

2. Download e Processamento dos Dados

Execute o script de carregamento de dados:

```
python src/1_data_loader.py
```

O que este script faz:

- Baixa o dataset **Sentiment140** do Hugging Face (1.6M de tweets)
- Filtra apenas as colunas `text` e `sentiment`
- Mapeia labels: `0` = Negativo, `4` → `1` = Positivo
- Salva o dataset processado em `data/sentiment140_processed.csv`

3. Análise Exploratória de Dados (EDA)

Abra e execute o notebook:

```
jupyter notebook notebooks/1.0-EDA.ipynb
```

Análises incluídas:

- ☒ Carregamento e inspeção dos dados
- ☒ Verificação de valores nulos e duplicados
- ☒ Distribuição de classes (balanceamento)
- ☒ Análise do comprimento dos textos
- ☒ Histogramas por sentimento
- ☒ **Word Clouds** (nuvens de palavras) para sentimentos positivos e negativos
- ☒ Exemplos de textos de cada classe

4. Treinamento do Modelo Baseline

Execute o script de treinamento:

```
python src/3_train_baseline.py
```

Pipeline do Modelo Baseline:

1. Carrega `data/sentiment140_processed.csv`
2. Aplica limpeza de texto com `clean_text()` :
 - Conversão para minúsculas
 - Remoção de URLs, menções (@) e hashtags (#)
 - Remoção de pontuação e números
 - Remoção de stopwords (inglês)
 - Lematização
3. Divide dados: **80% treino / 20% teste**
4. Treina pipeline: **TF-IDF + Logistic Regression**
5. Avalia com métricas completas
6. Salva modelo em `models/baseline_model.pkl`

Métricas reportadas:

- Accuracy Score
- Classification Report (Precision, Recall, F1-Score)
- Confusion Matrix

5. Fine-tuning de Modelos Transformer

Para treinar modelos SOTA usando DistilBERT:

```
python src/4_train_transformer.py
```

O que este script faz:

- Fine-tuning do modelo **DistilBERT-base-uncased**
- Tokenização especializada para Transformers
- Treinamento com **3 épocas** (configurável)
- Suporte a **GPU** (CUDA) com mixed precision (FP16)
- Métricas completas: Accuracy, F1, Precision, Recall
- Salva modelo em `models/sentiment_transformer/`
- Testa predições em exemplos

Accuracy esperada: ~85-90% ☒

Nota: Este processo pode levar várias horas em CPU. Recomenda-se usar GPU.

6. Fazer Predições

Use o script interativo para testar os modelos:

```
python src/5_predict.py
```

Você pode:

- Usar modelo **Baseline** (rápido)
- Usar modelo **Transformer** (mais preciso)
- **Comparar ambos** lado a lado

☒ Descrição dos Módulos

```
src/1_data_loader.py
```

Função principal: `load_and_save_data()`

Carrega o dataset Sentiment140 do Hugging Face, processa e salva localmente.

Exemplo de uso:

```
from src.data_loader import load_and_save_data, load_processed_data

# Baixar e processar dados
df = load_and_save_data()

# Ou carregar dados já processados
df = load_processed_data('data/sentiment140_processed.csv')
```

`src/2_preprocessing.py`

Função principal: `clean_text(text)`

Pipeline completo de limpeza de texto:

1. Minúsculas
2. Remoção de URLs
3. Remoção de menções e hashtags
4. Remoção de pontuação e números
5. Remoção de stopwords
6. Lematização

Exemplo de uso:

```
from src.preprocessing import clean_text

text = "I LOVE this product!!! Check it out: https://example.com @user #awesome"
cleaned = clean_text(text)
# Output: "love product check"
```

`src/3_train_baseline.py`

Função principal: `main()`

Script completo para treinar modelo baseline com pipeline do scikit-learn.

Componentes:

- **TF-IDF Vectorizer:** Converte texto em vetores numéricos
 - Max features: 10,000
 - N-grams: (1, 2) - unigramas e bigramas
 - Min/Max DF para filtrar termos muito raros/comuns
- **Logistic Regression:** Classificador linear
 - Regularização balanceada
 - Max iterations: 1000

Exemplo de uso:

```
# Treinar modelo
python src/3_train_baseline.py

# Ou importar e usar
from src.train_baseline import load_model, predict_sentiment

pipeline = load_model('models/baseline_model.pkl')
texts = ["I love this!", "This is terrible"]
predictions, probabilities = predict_sentiment(pipeline, texts)
```

`src/4_train_transformer.py`

Função principal: `main()`

Script completo de fine-tuning com DistilBERT.

Componentes:

- **Modelo:** DistilBERT-base-uncased (66M parâmetros)

- **Tokenizer:** Tokenização especializada com padding e truncation
- **Batch sizes:** 16 (treino), 64 (validação)
- **Épocas:** 3
- **Learning rate:** 2e-5 com warmup
- **Otimizações:** Mixed precision (FP16), gradient accumulation

TrainingArguments configurados:

- Evaluation strategy: por época
- Save strategy: por época (melhor modelo)
- Metric for best model: F1-Score
- FP16: Habilitado se GPU disponível

Exemplo de uso:

```
# Executar treinamento completo
python src/4_train_transformer.py

# Ou importar e usar
from src.train_transformer import load_model, predict_sentiment

# Carregar modelo treinado
tokenizer = AutoTokenizer.from_pretrained('models/sentiment_transformer')
model = AutoModelForSequenceClassification.from_pretrained('models/sentiment_transformer')
```

src/5_predict.py

Script interativo para fazer predições com modelos treinados.

Funcionalidades:

- Carregar modelo baseline ou transformer
- Fazer predições em novos textos
- Comparar ambos os modelos lado a lado
- Mostrar probabilidades e confiança

Exemplo de uso:

```
from src.predict import SentimentPredictor

# Usar modelo baseline
predictor = SentimentPredictor('baseline')

# Ou modelo transformer
predictor = SentimentPredictor('transformer')

# Fazer predições
texts = ["I love this!", "This is terrible"]
results = predictor.predict_with_labels(texts)

for res in results:
    print(f"Text: {res['text']}")
    print(f"Sentiment: {res['sentiment']} ({res['confidence']:.2f}%)")
```

src/utils.py

Funções utilitárias para salvar modelos, calcular métricas, plotar gráficos, etc.

📁 Dataset: Sentiment140

- **Fonte:** Hugging Face Datasets
- **Tamanho:** ~1.6M de tweets
- **Classes:**
 - 0 : Sentimento Negativo
 - 1 : Sentimento Positivo (originalmente 4, mapeado para 1)
- **Idioma:** Inglês
- **Tipo:** Tweets sobre diversos tópicos

📁 Resultados Esperados

Modelo Baseline (TF-IDF + Logistic Regression)

- **Accuracy esperada:** ~75-80%
- **Vantagens:** Rápido, interpretável, bom baseline
- **Limitações:** Não captura contexto semântico profundo

Modelos Transformer (RoBERTa, BERT)

- **Accuracy esperada:** ~85-90%
- **Vantagens:** Entendimento contextual, SOTA performance
- **Limitações:** Maior custo computacional, requer GPU

🔧 Tecnologias Utilizadas

- **Python 3.8+**
- **Scikit-learn** - Modelos clássicos de ML
- **NLTK** - Processamento de linguagem natural
- **Transformers (Hugging Face)** - Modelos pré-treinados
- **PyTorch** - Framework de deep learning
- **Pandas/NumPy** - Manipulação de dados
- **Matplotlib/Seaborn** - Visualizações
- **WordCloud** - Nuvens de palavras

🔧 Pipeline Completo

```
1. Download dos dados (Sentiment140)
   ↓
2. Análise Exploratória (EDA)
   ↓
3. Preprocessamento (clean_text)
   ↓
4. Split Treino/Teste (80/20)
   ↓
5. Treinamento
   ├── Baseline: TF-IDF + Logistic Regression
   └── SOTA: Fine-tuning Transformer
   ↓
6. Avaliação (Accuracy, F1, Confusion Matrix)
   ↓
7. Salvamento do Modelo
   ↓
8. Predições em Novos Textos
```

🔧 Testando o Modelo

Após treinar o modelo baseline, você pode testá-lo:

```
from src.train_baseline import load_model, predict_sentiment

# Carregar modelo
pipeline = load_model('models/baseline_model.pkl')

# Textos de teste
test_texts = [
    "I absolutely love this product! Best purchase ever!",
    "Terrible experience. Would not recommend to anyone.",
    "It's okay, nothing special but does the job."
]

# Fazer predições
predictions, probabilities = predict_sentiment(pipeline, test_texts)

for text, pred, prob in zip(test_texts, predictions, probabilities):
    sentiment = "POSITIVO 🟢" if pred == 1 else "NEGATIVO 🔴"
    confidence = prob[pred] * 100
    print(f"\nTexto: {text}")
    print(f"Predição: {sentiment} ({confidence:.1f}% confiança)")
```

📄 Próximos Passos

- ☐ Implementar validação cruzada
- ☐ Testar outros modelos (SVM, Random Forest, Naive Bayes)
- ☐ Otimizar hiperparâmetros com Grid Search
- ☐ Fine-tuning de modelos Transformer
- ☐ Deploy do modelo (Flask API, Streamlit)
- ☐ Análise de erros (onde o modelo falha?)
- ☐ Suporte para português (BERTimbau, dataset Multilingual)

📄 Contribuições

Contribuições são bem-vindas! Sinta-se à vontade para:

- Reportar bugs
- Sugerir melhorias
- Adicionar novos modelos
- Melhorar a documentação

📄 Licença

Este projeto é para fins educacionais.

Desenvolvido para análise de sentimentos em textos em inglês usando técnicas de NLP e Machine Learning.